R1.04 - Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Gestion des processus, multitâche préemptif et communication inter-processus via les signaux POSIX sous Unix/Linux

Synthèse exécutive : Ce cours pose les bases fondamentales des systèmes d'exploitation de type Unix/Linux en expliquant la transition d'un programme passif stocké sur disque à un processus actif en mémoire RAM. Il décrit le modèle multitâche préemptif géré par le noyau, la structure arborescente des processus (PID/PPID) dérivée du processus racine systemd/init (PID 1), le cycle de vie à trois états (Élu, Prêt, Bloqué), ainsi que le mécanisme universel de communication asynchrone entre processus au moyen des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme au Processus : Définitions, Attributs et Cycle de Vie

Pour exécuter plusieurs tâches de façon fluide, le noyau Linux s'appuie sur le multitâche préemptif : il attribue tour à tour de très courts intervalles de temps processeur (quanta de temps) aux différents processus. La création d'un nouveau processus s'effectue via l'appel système fork(), qui clone le processus parent, suivi généralement de exec(), qui remplace le code du processus enfant par un nouveau programme. Tous les processus forment un arbre généalogique strict dont le sommet est le processus initial systemd ou init (PID 1). Un processus passe dynamiquement par trois états principaux : il est 'Élu' lorsqu'il s'exécute sur le CPU ; si son temps est écoulé, l'ordonnanceur le repasse à 'Prêt' ; s'il demande une lecture sur disque ou une saisie clavier, le noyau le bascule à 'Bloqué' afin de ne pas gaspiller de cycles CPU pendant l'attente.

- Programme vs Processus : Un programme est un fichier binaire statique sur disque ; un processus est son exécution dynamique en mémoire RAM avec des ressources allouées.
- PID (Process Identifier) & PPID (Parent PID) : Identifiants uniques attribués par le noyau pour repérer chaque processus et tracer sa filiation.
- PCB (Process Control Block) : Structure de données interne au noyau conservant l'état du processeur, la mémoire, la priorité, l'UID et les descripteurs de fichiers ouverts.
- États d'un processus : Élu (exécute du code sur le CPU), Prêt (attend du temps CPU attribué par l'ordonnanceur), Bloqué (en attente d'une E/S ou d'un événement externe).

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme sur disque est comme un livre de recettes rangé dans une bibliothèque (passif, ne consomme rien). Le processus est le cuisinier qui prépare activement le plat en cuisine (dynamique, occupe de la place sur le plan de travail, utilise de l'énergie et des ustensiles).*

2. Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux représentent le canal de communication inter-processus (IPC) le plus élémentaire et asynchrone dans les systèmes Unix. Lorsqu'un signal est émis à destination d'un processus, le noyau suspend l'exécution normale du programme pour exécuter une fonction spécifique appelée 'gestionnaire de signal' (signal handler). Les développeurs peuvent intercepter (catch) ou ignorer (ignore) la plupart des signaux. Cependant, POSIX définit deux exceptions absolues : SIGKILL (9) et SIGSTOP (19). Ces deux signaux sont interceptés et traités directement par le noyau lui-même, garantissant à l'administrateur système un contrôle ultime sur tout processus hors de contrôle.

- Signal POSIX : Interruption logicielle envoyée à un processus pour l'informer d'un événement ou lui imposer une action.
- SIGINT (Signal 2) : Interruption déclenchée par 'Ctrl + C' pour interrompre gentiment une application interactive.
- SIGTERM (Signal 15) : Signal de terminaison propre envoyé par défaut par 'kill'. Permet au processus de sauvegarder et fermer ses ressources.

• SIGKILL (Signal 9) : Ordre d'arrêt inconditionnel exécuté directement par le noyau. Impossible à intercepter ou ignorer.

• Job Control (SIGSTOP/SIGCONT) : 'Ctrl + Z' envoie SIGSTOP pour suspendre un processus ; 'fg' ou 'bg' envoie SIGCONT pour relancer l'exécution.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe gentiment à la porte pour vous demander d'évacuer (vous avez le temps de ranger vos affaires et fermer à clé). SIGKILL (9) est l'intervention de la brigade de sécurité qui défonce la porte et vous dégage sur-le-champ sans vous laisser la possibilité de toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : Le tiret est obligatoire car il distingue l'option représentant le numéro du signal (`-9` pour SIGKILL) des identifiants de processus (PID). Sans le tiret, la commande `kill 9 1234` tente d'envoyer le signal par défaut (SIGTERM/15) aux deux processus dont les PID seraient 9 et 1234, ce qui échoue si le PID 9 n'existe pas ou n'appartient pas à l'utilisateur.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment le détruit-on avec la commande kill ?
Réponse du professeur : Un processus Zombie est un processus qui a terminé son exécution mais dont le père n'a pas encore lu le code de retour (via l'appel système wait). Il ne consomme plus ni CPU ni RAM, juste une entrée dans la table des processus. On ne peut pas le détruire avec 'kill' car il est DÉJÀ mort ! Pour le faire disparaître, il faut forcer son processus père à lire son état, ou tuer le processus père pour que PID 1 (systemd) l'adopte et le nettoie.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant tape la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Expliquez le rôle du symbole '&', détaillez ce qui se passe lors de la commande kill -15 et indiquez le résultat d'une vérification ultérieure avec ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' place l'exécution de la commande 'sleep 600' en arrière-plan (background). Le terminal affiche le job ID et le PID attribué (ex: PID 4521) et rend immédiatement la main à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' filtre la table des processus système et confirme la présence du processus sleep 600 sous le PID 4521 et identifie son PPID (le PID du shell Bash). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM au processus. Recevant SIGTERM, le processus sleep interrompt proprement son exécution et termine sa session.

Résultat : Le processus est arrêté proprement. Une exécution ultérieure de 'ps -ef | grep sleep' confirme la disparition complète du processus de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux ?

R : Le processus s'appelle 'systemd' (ou 'init' sur les anciens systèmes) et son PID est toujours 1.

Q : Quel signal POSIX est envoyé au processus en premier plan lorsqu'un utilisateur presse 'Ctrl + C' dans le terminal ?

R : C'est le signal SIGINT (Signal numéro 2).

Q : Pourquoi le signal SIGKILL (9) ne peut-il jamais être intercepté ou bloqué par un programme informatique ?

R : Parce qu'il n'est pas transmis au processus lui-même : il est intercepté et exécuté directement par le noyau Linux qui détruit le processus sans consulter sa table de gestionnaires de signaux.